

BLAST → DIAMOND: a concept for faster searches

Fast single-sequence comparison of a viral sequence against NCBI nr

As of 2026-09-23 — dell-3660 (192.168.5.23)

Generated by dell-3660:/home/gh/python/blast_diamond_pdf.py → /home/gh/diamond/tmp_prtest/pdf/blast_diamond_en.pdf

Contents

1	Starting point	1
2	Root cause	1
3	Strategy: DIAMOND instead of BLASTX	1
4	Status	1
5	Append: new sequences without a rebuild	2
6	Metadata in MariaDB (wagodb, prefix blast_)	4
7	Public API	6
8	AmyloDeep on the Tesla P4	10
9	Removing the duplicate storage	15
10	Possible next step	17

1. Starting point

blast2.py runs blastx for a single SARS-CoV-2 genome sequence (~30 kb, 6 frames) against the complete local NCBI nr database, restricted to fungi with **-taxids 4751**. Observed run time: > 20 minutes for a single run.

2. Root cause

Metric	Value
Database size	844 GB, 176 volumes
Sequences in nr	1.15 billion
Total residues	435 billion
RAM (dell-3660)	62 GB
Cores used by the script	8 of 20
GPU	Tesla P4, 8 GB VRAM, Pascal sm_61

The database is more than ten times larger than the available RAM, so the run is disk-I/O bound. The taxid filter reduces the reported hits but not the word search over the complete database. In addition, less than half of the CPU cores are used.

3. Strategy: DIAMOND instead of BLASTX

DIAMOND (Buchfink, Xie & Huson, *Nature Methods* 2015; extended in Buchfink et al. 2021, “Sensitive protein alignments at tree-of-life scale”) is built for comparing individual sequences against databases of nr size. Its core is double indexing

(query and database are sorted and seeded together instead of scanning the database linearly per query) plus spaced seeds. Reported speed-up over BLASTX: 100–20,000× at comparable sensitivity, and the database does not have to fit into RAM (block-wise streaming).

The same idea (a fast alignment prefilter against complete nr/nt to identify unknown or novel pathogens) is behind CZ-ID / IDseq (Kalantar et al. 2020, *GigaScience*), a pipeline of the Chan Zuckerberg Biohub for exactly this use case.

Alternative: MMseqs2 (Steinegger & Söding, *Nature Biotechnology* 2017) follows the same prefilter idea; at maximum sensitivity it is slightly slower than DIAMOND but sometimes more accurate for distant homologues. DIAMOND was chosen: least migration effort (drop-in for blast2.py), no additional infrastructure.

4. Status

Instead of a physical subset (fungi) there is a complete `nr_full.dmnd` with taxonomy map; searches are restricted with `-taxonlist`. The build runs on `sda5 (/mnt/archive)`.

One pass for .dmnd and metadata: `~/python/nr_build.py` (since 2026-09-23, replaces `diamond_pipeline.sh`, `filter_nonempty.py`, the initial import of `blast_meta2db.py` and `blast_release_finalize.sh`). A single `blastdbcmd` stream (per define OID, accession, TaxID, length, sequence, title) is split: to `makedb` with headers made of accessions only (`>acc1 >acc2 ...`, titles live only in MariaDB), and to MariaDB as chunk files of 5 million sequences loaded by a separate thread (bounded queue: it slows the stream instead of filling the disk). Empty sequences are left out of the database file and recorded in `blast_dmnd_luecke`. Then both targets are verified against `blastdbcmd -info`, indexes are built (`tmpdir` on the NVMe), taxonomy and `blast_release` are updated. Only then does the API accept searches.

Memory failure in the first build (2026-09-23, 11:03): `makedb` collected the offset table (16 bytes per sequence) in a vector (`dmnd.cpp`) that doubled its capacity at 2^{30} entries – volume ~164 of 176 – and briefly needed ~51 GB; the OOM killer ended the build after 3.5 h. Patch: entries go to `<db>.pos_tmp` in blocks of one million and are copied into the trailer at the end. Checked on 2.5 million nr sequences: file byte-identical to the unmodified binary, `--append` identical as well. Builds use `~/src/diamond/build/diamond (v2.2.8 + --append + this patch)`, the same binary the cron scripts use.

A BLAST-formatted nr in `~/blastdb` no longer exists. The API (`acc`, section 7.7) and AmyloDeep (`--acc/--kandidaten`, section 8) fetch residues directly from the `.dmnd` via `nr_seq.py (/home/gh/diamond/nr_full.dmnd, NVMe)`; regions that `makedb` masked to X are fetched from NCBI when needed. Section 9 describes the state as of 2026-09-24.

4.1 Throughput of the initial build

Measured on the build of 2026-09-23 11:34, read from `blast_import` and `nr_build.log` when this PDF was generated. A single `blastdbcmd` stream feeds `makedb` (20 threads) and the loader thread at the same time, so these figures cover both targets together, not `makedb` alone.

Quantity	Value
Elapsed	171:44 h (11.3 h in total)
Sequences	1,156,964,465 (100.3 % of nr)
Defines (<code>blast_acc</code>)	1,943,564,203
Sequences/s	1,871
Defines/s	3,143
Residues/s (derived)	0.7 M, mean length 377 aa
<code>nr_full.dmnd</code>	489.5 GB, 1 MB/s
Chunk (5 M sequences) into MariaDB	81 s (0–107 s)
Loader thread busy	3.4 % of the elapsed time
Machine CPU (sar, 16:20:02)	8.8 % user, 0.0 % nice, 0.7 % system, 0.4 % iowait, 90.1 % idle
Load average (1/5/15 min)	8.33 / 9.60 / 6.10 on 20 threads
Empty sequences	0

The loader thread is busy 3.4 % of the elapsed time – most of it spent waiting for MariaDB. `makedb` memory stays flat across the whole build, which is the point of the patch in section 4.

MariaDB is the limit, not DIAMOND. A measurement over 5 s (`/proc/<tid>/stat` and `wchan`, 2026-09-23 16:11) found every stage but one waiting: `nr_build.py` 0 % in `futex_do_wait` on the full queue, the loader thread 0 % in `wait_woken` on the MariaDB socket, `makedb` 0 % in `anon_pipe_read`, `blastdbcmd` 0 % in `anon_pipe_write`. Only

mariadb was running, at 100 %, on a single `LOAD DATA LOCAL INFILE`. The queue agrees: it is full, the chunk files sit ready on the NVMe and the stream is throttled – exactly as intended.

That is why the machine lies largely idle, and the disks with it (NVMe 8 %, sda 11 % utilisation, `iostat`, at the same time). The nice share in the table above is not the build but the AmyloDeep worker of section 8, computing the control run alongside. A single connection writes `blast_seq` and `blast_acc`, both `ROW_FORMAT=COMPRESSED`: every page is compressed as it is written, and that is what costs the core. Secondary indexes are built afterwards and play no part yet. The 20 threads of `makedb` have no effect in this phase – they hang on a pipe fed by a single Python process. So the build would get faster on the metadata side, not on DIAMOND: several loader connections on disjoint OID ranges, or uncompressed tables compressed later. Neither has been measured.

5. Append: new sequences without a rebuild

NCBI offers no incremental update for the BLAST-formatted `nr` (README `blast/db`: “Incremental update is not available”), and DIAMOND cannot append to an existing `.dmnd`. Both were added.

5.1 Patch `diamond makedb --append`

Branch `makedb-append` in `~/src/diamond`, diff in `~/src/diamond-makedb-append.patch` (3 files, ~140 lines), binary `~/src/diamond/build/diamond` (v2.2.8 + patch). Layout of a `.dmnd`: header, sequences, then the trailer with offset table, taxon lists (varint-encoded per OID, no index of their own), taxonomy nodes and names; the header is written last via `seek(0)`. What `--append` does:

1. Read and check the header (magic, version 3, same use of `--taxonmap/--taxonnodes/--taxonnames` as the initial build).
2. Back up header and complete trailer to `<db>.append_backup` (about 20–25 GB for `nr`).
3. Truncate the file at the end of the sequences (`pos_array_offset`) and write the new sequences after them. The running MurmurHash3 continues from the stored value.
4. Rewrite the trailer: old offset entries + new ones, old taxon lists byte for byte + new ones (`TaxonList::build` only for the new OIDs), taxonomy from the current `taxdump`.
5. Write the header, delete the backup. On any error the patch restores header and trailer from the backup.

Tests (2026-09-23): (1) a DB built from A (12,000 sequences) plus `--append` of B (8,000) is **byte-identical** to the DB built from A+B in one pass (`cmp`), including hash, taxon lists and header. (2) Appending a sequence of length 0 aborts; the DB is byte-identical to its previous state, no backup left over. (3) `diamond blastp` with `staxids` output on the extended DB returns correct hits.

5.2 Update script `diamond_append.py`

Detecting a new release: NCBI's `nr-prot-metadata.json` (last-updated, number-of-sequences) is compared with the newest row in `blast_release`; equal means done. For a new release: download volume by volume (md5-checked), unpack, check accessions against a sorted 64-bit hash array of all known accessions (~15 GB, memory-mapped), delete the volume. Peak disk use is one volume instead of 855 GB. A sequence is new if none of its accessions is known. Then exactly one `--append` (a state file prevents appending twice), metadata in `blast_seq/blast_acc` with new OIDs from the previous maximum, a new row in `blast_release`, mail to `gh@heissa.de`. Resumable per volume. When a sequence was added is available from the views `blast_seq_herkunft/blast_acc_herkunft` (OID range in `blast_import` → release).

End-to-end test with test tables `blasttest_*`: existing set A, “new release” = volume `nr.000` up to sequence 20,000. Result: exactly 8,000 new sequences detected, `.dmnd` byte-identical to the full DB afterwards, metadata checksum-identical to production, a second run reports “no new release”.

Limits: new accessions on existing sequences (a new organism for the same sequence) and sequences withdrawn by NCBI are only counted and handled at the next full rebuild. 64-bit hash: an expected ~0.2 accessions per release comparison are wrongly classified as known.

Sequences already appended by the daily delta (5.3) are recognised by their sequence hash and not appended a second time (counter “already via daily delta”). The MariaDB part runs in one transaction (6.4). Test with test tables: 3,000

sequences, 1,000 of them already in nr (accession) and 10 already via daily delta (sequence) → exactly 1,990 appended, old release rows unchanged.

5.3 Daily delta nr_daily_delta.py

NCBI rebuilds the BLAST nr only as a whole (latest 2026-09-16; blast/db/FASTA/nr.gz frozen since 2024-02-06). Between releases, new proteins come from the daily files: genbank/daily-nc/ncMMDD.gnp.gz (GenPept, kept ~180 days) and refseq/daily/rsnc.MMDD.YYYY.gpff.gz (~21 days). GenPept contains accession, title, TaxID (/db_xref="taxon:"), sequence and, in the LOCUS line, the NCBI date.

Duplicates by sequence, not by accession: nr merges identical proteins into one entry but does not list every accession (example: YGQ27362.1 has the same sequence as YP_009724396.1, whose nr entry has only 7 defines). Matching by accession would therefore append duplicates en masse. Instead: a blake2b-64 hash of every sequence (upper case) is checked against the sorted hash array of all nr sequences (/mnt/archive/nr_seq_hashes.npy, 9.2 GB, memory-mapped) plus those already seen in the same run. All pending daily files are processed in **one** --append; the TaxID mapping comes as a small mapping file from the GenPept itself.

Test (2026-09-23, test tables, test dmnd): nc0919 (Sep 19): 5,189 proteins, 100 already in the test "nr", 3,195 duplicated within the file, 1,894 new; nc0920: 7,104 proteins, 6,491 new. The dmnd then held 100 + 8,385 sequences, OIDs in MariaDB without gaps, every new sequence with its NCBI date; a second run reports "no pending daily files". Example of a new human sequence: BIN93201.1 *spexin precursor, partial [Homo sapiens]*, NCBI date 2026-09-19.

Cron: 03:30 diamond_append.py (new nr release), 04:30 nr_daily_delta.py (daily files). Both take the same lock /home/gh/.nr_dmnd.lock and wait until the initial build (dmnd_hash and meta_importiert in blast_release) is complete. After a new nr release the daily delta starts from its date; sequences appended by the delta in the meantime are skipped by the release comparison via the sequence hash.

Robustness (since 2026-09-30): (1) If diamond_append.py is still running at 04:30 (a new nr release takes hours), the delta waits up to 12 h for the lock (LOCK_WAIT_H) instead of skipping the day. (2) The hash caches are extended block by block (merge_in_datei: old file memory-mapped, new one via open_memmap, blocks of 2²⁶ entries) instead of np.union1d in RAM – for nr_acc_hashes.npy (15.5 GB) that needed about 47 GB and ended in an OOM kill on 2026-09-30; now ~0.8 GB of anonymous memory. (3) If a run finds an old ~/nr_daily/state.json whose daily files are all in blast_release already, the metadata commit went through and the run died only afterwards: it adds the sequence and accession hashes from the leftover .fa/.acc.tsv (idempotent) and clears the directory – even when no new daily files are pending. If metadata is missing, it still stops with an error (dmnd and MariaDB could diverge).

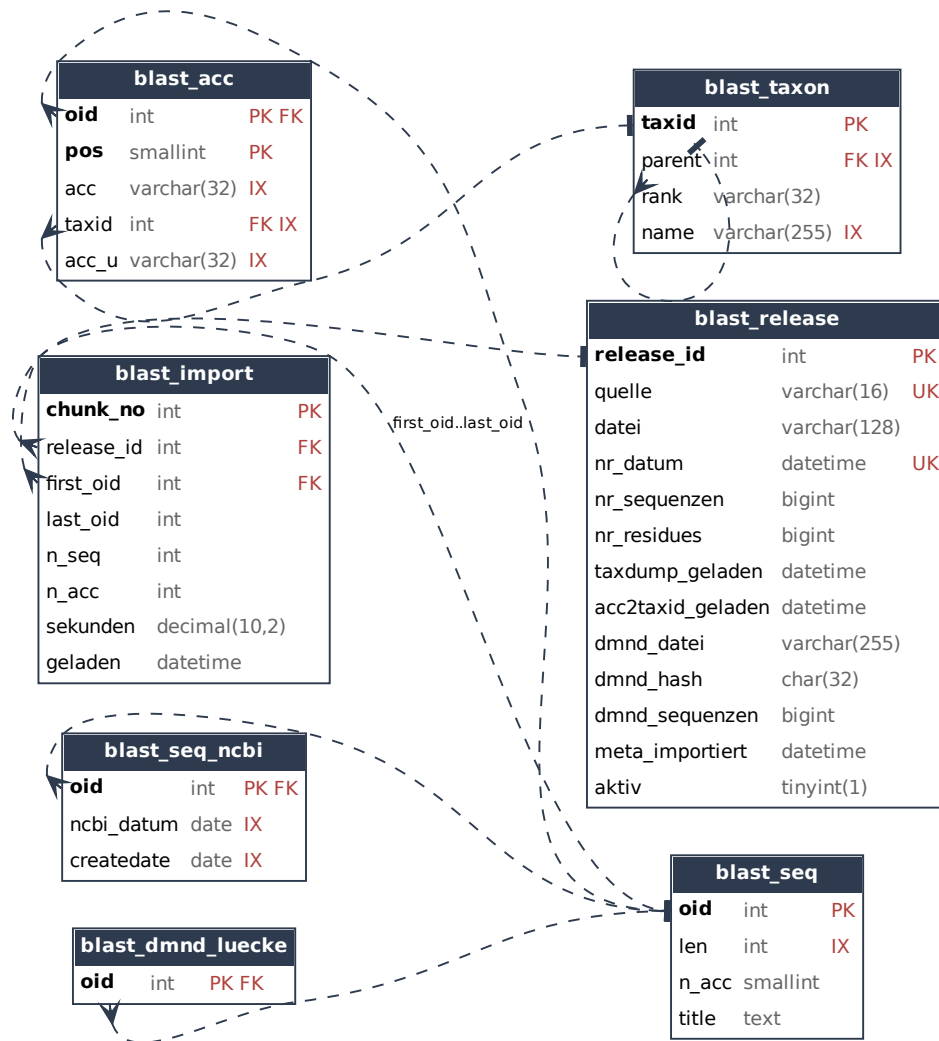
6. Metadata in MariaDB (wagodb, prefix blast_)

The .dmnd file only knows sequences, accessions as names and TaxIDs – no date, no NCBI release, no provenance. The BLAST nr has no submission date either. Questions like "which nr release is loaded", "since when is this sequence in the DB" or "when was the last human sequence added" can therefore only be answered from the metadata in MariaDB. They are also the basis of the API (DIAMOND hits enriched by accession with title, organism and taxonomy). Every script that changes the .dmnd updates these tables in the same run.

6.1 Tables

Table	Rows (nr 09/2026)	Content
blast_seq	1.15 billion	per sequence: oid (PK), length, number of accessions, title of the first define
blast_acc	~1.94 billion	per define: oid, pos, acc, taxid; indexes on acc and taxid
blast_taxon	~2.7 million	NCBI taxonomy: taxid, parent, rank, scientific name
blast_release	1 + 1 per daily file	release: quelle (nr / genbank-daily / refseq-daily), datei, nr_datum, sequences, dmnd_hash, aktiv
blast_import	231 + 1 per delta	OID range first_oid..last_oid → release_id, load time
blast_seq_ncbi	delta sequences only	oid → NCBI date from the GenPept LOCUS line

blast_seq and blast_acc are InnoDB ROW_FORMAT=COMPRESSED. At an intermediate import state (140 million sequences) each used 6.3 GB; extrapolated to all of nr ~52 GB each, plus the two indexes on blast_acc. Everything lives on the system NVMe (/var/lib/mariadb).



ER diagram of the nr metadata, generated from the live schema (information_schema). Solid: declared foreign keys, dashed: logical links; crow's foot = many. PK/UK/IX/FK as in the database; indexes on blast_acc appear once nr_build.py has built them.

6.2 Provenance and date per sequence

Through its OID range in blast_import, every sequence belongs to exactly one row in blast_release. The views blast_seq_herkunft and blast_acc_herkunft resolve this: quelle, datei, nr_stand, ncbi_datum, hinzugefuegt (import time). Limit: all sequences of the initial import only carry the nr release 2026-09-16, because the BLAST nr has no submission date. Only sequences from the daily delta have a real date (ncbi_datum from the LOCUS line).

OID numbers: the OIDs in MariaDB are the BLAST OIDs, continued for appended sequences. They do not match the record number in the .dmnd (empty sequences were dropped during the build). The link is therefore the accession (DIAMOND output sseqid → blast_acc.acc).

6.3 Example queries

```
-- which release is loaded?
SELECT quelle, datei, nr_datum, nr_sequenzen, dmnd_hash
FROM blast_release ORDER BY nr_datum DESC LIMIT 5;

-- when was the last human sequence added?
SELECT acc, quelle, datei, ncbi_datum, hinzugefuegt
FROM blast_acc_herkunft WHERE taxid = 9606
ORDER BY hinzugefuegt DESC, ncbi_datum DESC LIMIT 1;

-- new sequences per day
SELECT n.ncbi_datum, COUNT(*) FROM blast_seq_ncbi n GROUP BY 1 ORDER BY 1 DESC;

-- enrich a DIAMOND hit
SELECT a.acc, s.title, t.name FROM blast_acc a
JOIN blast_seq s USING (oid) JOIN blast_taxon t ON t.taxid = a.taxid
WHERE a.acc = 'YP_009724390.1';
```

6.4 Who writes what

Script	When	Writes
blast_meta2db.py	once (initial build)	blast_seq, blast_acc in 231 chunks of 5 million sequences (LOAD DATA LOCAL, ~95 s per chunk, resumable via blast_import), then indexes and blast_taxon
blast_release_finalize.sh diamond_append.py	after build + import cron 03:30, new nr release	dmnd_hash, dmnd_sequenzen, meta_importiert for the initial release new blast_release row (quelle nr), OID range, blast_seq/blast_acc of the new sequences
nr_daily_delta.py	cron 04:30, daily files	one blast_release row per daily file, OID range, blast_seq/blast_acc/blast_seq_ncbi

Consistency .dmnd ↔ MariaDB: the append scripts first append to the .dmnd and record that in a state file (dmnd_done, hash, sequence count). Then they write all metadata in **one** transaction. If that part fails, MariaDB rolls back completely; the next run does not append again but only adds the missing metadata. Tested: with a deliberately missing table, MariaDB stayed at the old state, the .dmnd was not extended twice (hash unchanged), and the follow-up run added exactly the missing 8,385 sequences. The hash arrays (accessions, sequences) are only updated after the commit.

Credentials: all BLAST scripts get them via ~/python/blast_db.py (reads DB_CFG from ~/mqtt-listener.py, database wagodb).

6.5 Character-set trap in joins on blast_acc.acc

blast_acc.acc is ascii, tables like amy1_sequenz.acc are utf8mb4. MariaDB then converts the blast_acc column and cannot use idx_acc: a full scan over 1.9 billion rows, and the view amy1_v_ergebnis hangs. With a.acc = CONVERT(s.acc USING ascii) the join uses the index (344 hits in 0.003 s). Permanently, a virtual column acc_u VARCHAR(32) CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci AS (acc) VIRTUAL with index idx_acc_u extends the table (script blast_acc_utf8mb4_index.sh, two online ALTER TABLEs; build started 2026-09-24 08:49, takes hours). After that utf8mb4 tables join with a.acc_u = s.acc without CONVERT.

7. Public API

So that others can use the DIAMOND search and the metadata, there is a minimal JSON API: <https://yt.heissa.de/blast/api.php>. yt.heissa.de only has an AAAA record, so from outside the API is reachable **over IPv6 only** (tested from heissa.de). Without parameters it returns its own description.

7.1 Endpoints

Call	Access	Returns
GET ?r=release	open	loaded nr release, daily deltas since, readiness of search and metadata, queue
GET ?r=release → ram	open	free memory (MemAvailable), limit for one search, measured k per mode/sensitivity (section memory planning)
GET ?r=acc&acc=YP_009724390.1	open	title, length, provenance/date of the sequence, TaxID with name, all identical entries (up to 200). Without .version: latest version
GET ?r=taxon&taxid=9606 or &name=Homo sapiens	open	taxon with its complete lineage
POST ?r=search	X-API-Key	creates a search job (query via seq or acc); response 202 with job_id, position and result URL
GET ?r=job&id=...	anyone with the id	status, parameters, run time, nr release/dmnd_hash of the search; when done, the hits with title and taxon names from MariaDB

Search parameters (form or JSON): seq (FASTA or raw sequence, up to 50 sequences / 100,000 letters) **or** acc (accession from nr, optionally with range from-to, blastp only; see 7.7), mode blastp or blastx, taxonlist (up to 20 TaxIDs including sub-taxa, e.g. 4751 = fungi, 9606 = human), len_min/len_max (length of the hit sequence), new_days (created at NCBI within the last N days), eval (default 0.001), max_target_seqs (1–500, default 25), sensitivity fast to ultra-sensitive (default sensitive). The result additionally contains preselect, block_size, ram_est_gb, ram_gb (measured) and ram_free_gb.

Language (since 2026-09-30): responses, field names and error messages of the API are English (endpoints, daily_deltas, ready, sequence, lineage, created/started/finished, result, windows ...). The former German parameters neu_tage and von/bis are still accepted (now new_days, from/to); the database columns keep their names and get English aliases in the queries.

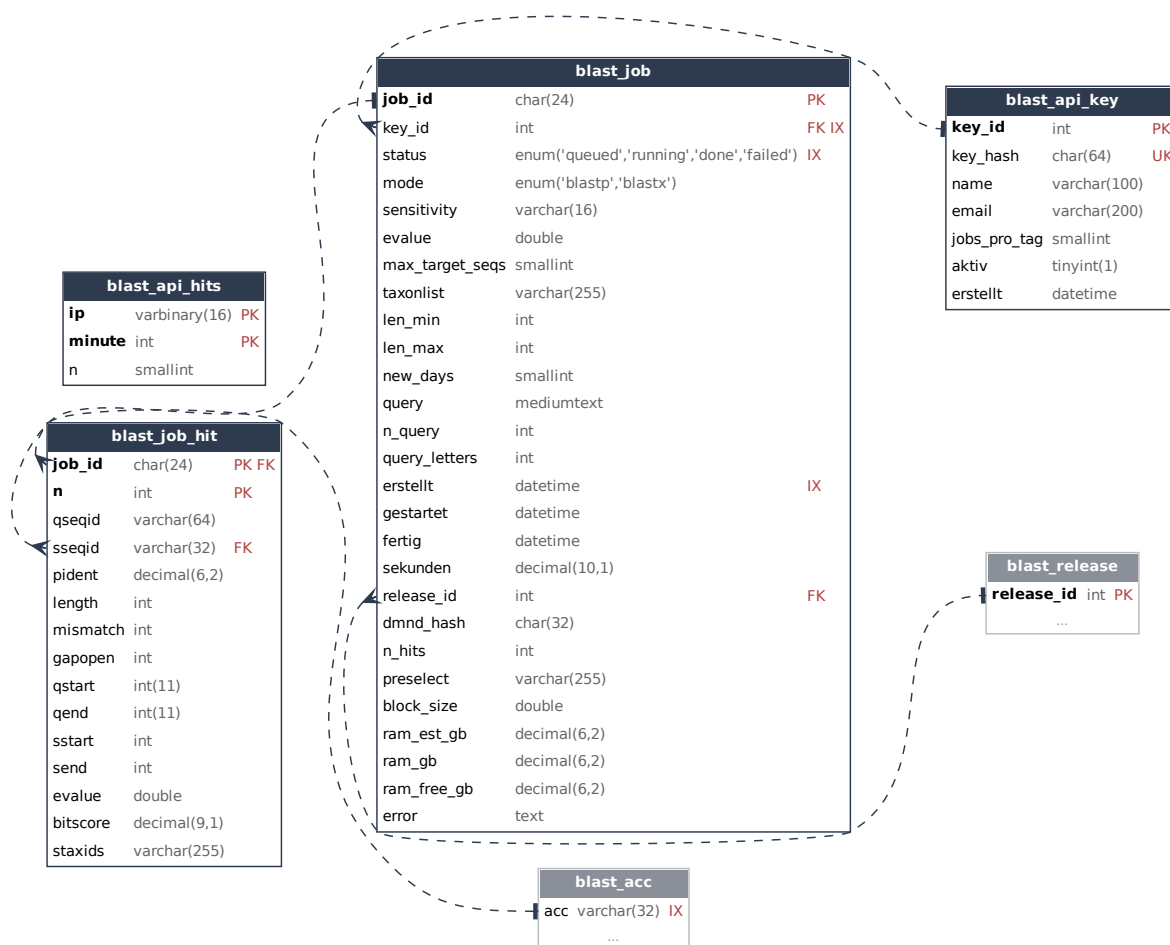
```
curl -X POST -H 'X-API-Key: ...' -H 'Content-Type: application/json' \
  -d '{"seq": ">q1\nMKT...", "mode": "blastp", "taxonlist": "9606"}' \
  'https://yt.heissa.de/blast/api.php?r=search'

curl 'https://yt.heissa.de/blast/api.php?r=job&id=18a8ceae9fcc7d63063e5ffc'
```

7.2 Architecture

api.php (/var/www/blast, mod_php, alias in the yt.heissa.de vhost in front of the Invidious proxy) answers metadata requests directly and only records searches as a row in blast_job. For acc it first fetches the query sequence with nr_seq.py ACC [--range from-to] from the .dmnd (direct access via the offset table, ~0.1 s; section 9). A DIAMOND search against full nr occupies all 20 cores for minutes and must never run inside the web server process. **blast_api_worker.py** (systemd blast-api-worker, user gh) takes the jobs one by one, runs DIAMOND (time limit 1 h), writes the hits to blast_job_hit and records nr release and dmnd_hash. During a search it holds a shared lock on /home/gh/.nr_dmnd.lock; the append scripts need the exclusive lock and therefore never change the .dmnd under a running search. After a crash it resets stuck jobs to queued. As long as dmnd_hash is missing in the active release (initial build), the API rejects searches (503).

Table	Content
blast_api_key	key as sha256 only, name, e-mail, daily quota, active
blast_job	job: parameters, query, status, timestamps, run time, release_id, dmnd_hash, hit count, error
blast_job_hit	hits in DIAMOND format 6 (qseqid ... bitscore, staxids)
blast_api_hits	counter per IP or IPv6 /64 and minute (rate limit), cleaned up by the worker
blast_ram_log	peak RSS of every search (mode, sensitivity, query and DB letters, block_size, estimate, free memory) – basis of the memory estimate
blast_ram_status	one row: MemAvailable, MemTotal, SwapFree; written by the worker every 30 s



ER diagram of the API (grey: tables from section 6, linked columns only), generated from the live schema (information_schema). Solid: declared foreign keys, dashed: logical links; crow's foot = many. PK/UK/IX/FK as in the database.

7.3 Pre-selection via MariaDB (dmnd_preselect.py)

Apart from the offset table (16 bytes per sequence, pos_array_offset) the .dmnd has no index; DIAMOND allows a seed index (diamond makeidx, .seed_idx) only up to 100 million letters (MAX_LETTERS in data/index.cpp), nr has 436 billion. Every search therefore builds the seeds anew and reads the whole file – even with --taxonlist. It only gets faster when fewer sequences are searched: if taxonlist, len_min/len_max or new_days is set, MariaDB determines the OIDs (taxon including sub-taxa via a recursive CTE over blast_taxon.parent and blast_acc.idx_taxid; length via blast_seq; creation date via blast_seq_ncbi.createdate), dmnd_getseq.Dmnd reads the records via the offset table, and diamond makedb builds a sub-dmnd in ~/diamond/preselect (cached 2 days, key = criteria + dmnd_hash). The search uses --dbsize = letters of the full nr so that E-values stay comparable with a full search; the worker adds the hits' taxa from blast_acc. Above 50 million sequences (PRESELECT_MAX_SEQ) the sub-dmnd no longer pays off: then a full search with --taxonlist and a post-filter on length/date.

Pre-selection	Sequences	Build	Search TTR 20–70
none (full nr)	1.156.964.465	–	895 s (fast, -b 2)
new_days=90	2.543.725	31 s	1.8 s fast / 12.9 s sensitive
new_days=4	71.699	1,0 s	–

Limit of the date filter: createdate exists only for sequences appended by the daily delta (from 2026-09-16) – not for the 1.15 billion sequences of the nr release, and the BLAST database itself holds no per-sequence date. Backfill for the nr release: blast_ncbi_backfill.py reads the GenBank daily files from 2026-07-02 (3.25 million OIDs of the nr release), then blast_createdate_fill.py --von-vorn fetches the creation date via esummary. RefSeq daily files only go back ~4 weeks.

7.4 Memory planning for every search

DIAMOND loads the database in blocks of `--block-size` billion letters; each shape (number depends on the sensitivity) gets a seed table of ~18 bytes per seed. For small queries DIAMOND filters the block's seeds against the query's seeds (query_seeds_hashed), so the need depends strongly on the query size. The `.dmnd` only provides the letter count (header, letters); the rest is measured. Before every search the worker computes:

1. `limit = min(48 GB, MemAvailable - 4 GB reserve)`, read from `/proc/meminfo` right before the start.
2. `need = 0.5 GB + k · min(DB letters, b)` (in billions). `k` (GB per billion letters) = largest measured value in `blast_ram_log` with the same mode and sensitivity and a query within `q/4...4q`, else from larger queries, else a start value; plus 20 % margin.
3. `b` = the largest step of `2 / 1 / 0.5 / 0.25 / 0.1` whose need fits under the limit (fewer blocks = faster). If none fits, the worker waits up to 30 min for free memory, then fails.
4. DIAMOND runs under `/usr/bin/time -f %M`; the peak RSS goes, with estimate and free memory, into `blast_ram_log` and into the job (`ram_gb`). Every search improves the next estimate.

Start values `k`: fast 1.5, sensitive 3, mid-sensitive 3, more-sensitive 5, very-sensitive 8, ultra-sensitive 12; $\times 2$ for queries above 5,000 letters and for `blastx`. fast and sensitive come from measurements (block test 2026-09-24, sub-.dmnd 2026-09-30), the others are cautious assumptions until measurements exist. Apache cannot see `/proc/meminfo` (systemd `ProcSubset=pid`), so the worker writes the memory state for `?r=release` to `blast_ram_status` every 30 s. Since 2026-09-30 the dell also has 32 GB of swap on the NVMe (`/swapfile`, `vm.swappiness=10`) as a buffer against OOM kills.

Test (2026-09-30, external via heissa.de): SARS-CoV-2 spike YP_009724390.1 as S1 (residues 14–685) and S2 (686–1273), 1,260 aa, ultra-sensitive, `new_days=100`. Pre-selection 2,544,520 sequences / 1.08 billion letters in 37.3 s (OIDs 5.5 s, records 27.5 s, makedb 4.3 s), search 46 s, job total 113.8 s. Memory: 52.3 GB free, estimate 16.1 GB (start value), measured 1.88 GB at `--block-size 2 - k = 1.27` GB per billion letters is stored in `blast_ram_log` since. Without a hit limit (directly on the dell, 57.6 s): S1 506 hits ($265 \geq 90$ % identity), S2 510 hits ($382 \geq 90$ %).

7.5 Protection

Dedicated MariaDB user `blast_api`: SELECT only on the metadata tables and views, INSERT only on `blast_job`, INSERT/UPDATE only on `blast_api_hits`, SELECT on `blast_ram_log/blast_ram_status`; credentials in `/etc/blast_api.ini` (root:www-data, 640). All queries use prepared statements, inputs are pattern-checked; `nr_seq.py` is called without a shell and with a validated accession/range. 60 requests per minute and client. Searches only with a key, at most 3 open jobs per key, daily quota per key (default 20). The `job_id` is 96 random bits and at the same time the permission to read the result. Accession requests return 503 until the index `idx_acc` exists – without it every request would scan 1.9 billion rows.

Managing keys (on the dell): `python3 ~/python/blast_api_key.py add NAME [EMAIL] [JOBS_PRO_TAG]` prints the key once; `list` shows all keys with their job count; `off KEY_ID` disables one.

7.6 Test (2026-09-23)

From outside (heissa.de, IPv6): description, release and the error cases (missing/wrong key, invalid sequence, invalid id, missing index) return the expected codes. Search path end to end against a test dmnd (daily file nc0920, 6,491 sequences): `blastp` of the human sequence BIN93201.1 with `taxonlist=9606` → 1 hit, 100 % identity, E-value $3e-72$, 0.5 s. Titles and taxon names appear once the metadata import has built the index and loaded the taxonomy. Test data removed afterwards. The `acc` path has not yet run end to end: while `diamond makedb` is running, searches return 503.

7.7 Example: SARS-CoV-2 spike RBD against human proteins

Question: does the receptor-binding domain (RBD) of the spike protein resemble human proteins? Only the **query** is sent, i.e. the sequence for which hits are searched. The subject side is always the complete `nr` (`nr_full.dmnd`); `taxonlist=9606` restricts hits to *Homo sapiens*, `mode=blastp` compares protein against protein.

Option A – query by accession. The spike sequence itself is in `nr` (YP_009724390.1, 1,273 aa). The RBD is residues 319–541 (223 aa); the API cuts it out with `range`, the query is then named `YP_009724390.1_319-541`:

```
curl -X POST -H 'X-API-Key: YOUR_API_KEY' -H 'Content-Type: application/json' \
-d '{"acc": "YP_009724390.1", "range": "319-541", "mode": "blastp",
"taxonlist": "9606", "max_target_seqs": 25}' \
'https://yt.heissa.de/blast/api.php?r=search'
```

Option B – query as a sequence, e.g. for your own variant or one not yet in nr. The same RBD as FASTA; line breaks within the sequence are allowed:

```
cat > rbd.fa <<'EOF'
>SARS-CoV-2_Spike_RBD
RVQPTESIVRFPNITNLCPFGEVFNATRFASVYAWNKRKISNCVADYSVLVNSASFSTFK
CYGVSPPTKLDLCFTNVYADSFVIRGDEVQRQIAPGQTGKIADYNYKLPDDFTGCVIAWNS
NNLDSKVGGMNYLYRLFRKSNLKPFRDISTEIYQAGSTPCNGVEGFNCYFPLQSYGFQ
PTNGVGYPYRVVVLFSFELLHAPATVCGPKKSTNLVKNKCVNF
EOF
curl -X POST -H 'X-API-Key: YOUR_API_KEY' \
--data-urlencode seq@rbd.fa -d mode=blastp -d taxonlist=9606 -d max_target_seqs=25 \
'https://yt.heissa.de/blast/api.php?r=search'
```

JSON works as well; the sequence then goes on one line in the seq field (FASTA header followed by \n). seq and acc together are rejected (400); an accession that is not in the BLAST nr of 2026-09-16 returns 404 – use option B then.

Response and result: the POST returns 202 immediately with job_id, queue position and result URL; poll it until status = done:

```
{ "job_id": "...", "status": "queued", "position": 1,
"url": "https://yt.heissa.de/blast/api.php?r=job&id=..." }

curl 'https://yt.heissa.de/blast/api.php?r=job&id=...'
```

Each hit contains the DIAMOND columns (sseqid, pident, length, qstart–qend, sstart–send, evalue, bitscore) plus title and taxon names from MariaDB. E-value and identity over the aligned length are what matters; short hits with an E-value close to 0.001 are usually chance.

8. AmyloDeep on the Tesla P4

A second application on the same sequences: predicting amyloid propensity with protein language models. AmyloDeep 0.3.1 (Davtyan, Khachatryan & Petrosyan, bioRxiv 2025, doi 10.1101/2025.09.16.676495; PyPI amylodeep, weights AlisaDavtyan/amylodeep-models on Hugging Face) is an ensemble of five models: ESM-2 150M (fine-tuned), UniRep with a classification head, ESM-2 650M (mean embedding) with a head, and SVM and XGBoost on the same 650M embeddings. The individual probabilities are calibrated (Platt or isotonic) and averaged. Scoring uses a rolling window (default 10 residues, step 1); each window is padded to 128 tokens.

8.1 Does it fit on the P4?

Model	Weights FP32	on the P4
ESM-2 150M (fine-tuned)	~0.6 GB	yes
ESM-2 650M (esm2_t33_650M_UR50D)	~2.6 GB	yes
together, plus llama-server (0.8 GB)	~4.1 GB of 7.5 GB	yes, FP32, no FP16 needed
for comparison: ESM-2 3B	~11.4 GB (FP16 ~5.7 GB)	no, or only without llama-server

UniRep runs via JAX on the CPU, as do SVM and XGBoost; the Torch models live on the GPU.

8.2 Environment: env iver with CUDA 11.8

The micromamba env iver (~/.iver_sim/mamba/envs/iver) was pinned to CUDA 11.8 but only contained a **CPU build** of PyTorch 2.5.1. It was replaced by the conda-forge build pytorch 2.5.1 cuda118, whose architecture list includes sm_61 natively, so no custom compile was needed. cudnn, magma and nccl came along; libprotobuf 5.28.3 → 5.28.2.

The next install would immediately have pulled PyTorch back to 2.10 CPU and numpy to 2.x. conda-meta/pinned therefore now also contains pytorch 2.5.1 cuda118*, libtorch 2.5.1 cuda118* and numpy 1.26.* (backup

pinned.bak-20260923). Installed: transformers 5.16.1, huggingface_hub 1.32, jax/jaxlib 0.5.2 (CPU), xgboost 3.4.2 (CPU), equinox, optax, optuna; amylodeep 0.3.1 and jax-unirep 3.0.0 via pip with --no-deps. openmm.testInstallation still passes (CUDA and OpenCL).

Executable stack: kernel 7.0 with glibc 2.43 no longer loads libraries that request an executable stack (cannot enable executable stack). Only jaxlib/xla_extension.so was affected: the X bit in its PT_GNU_STACK program header was cleared (RWE → RW); the original is kept as ~/iver_sim/xla_extension.so.orig. This has to be repeated after a jaxlib update.

8.3 amylo_gpu.py

~/iver_sim/ol_amyloid/amylo_gpu.py subclasses EnsembleRollingWindowPredictor without modifying the package. The package itself only runs on the CPU and evaluates ESM-2 650M three times for the same windows (NN head, SVM, XGBoost). The wrapper moves the Torch models to CUDA, works in batches of 64 and computes the 650M embeddings only once per set of windows. Input: sequence, FASTA, --acc or --kandidaten (TSV accession, short name); sequences come from the .dmnd via nr_seq.py, X masking resolved via NCBI (see section 9).

Check against the original (--vergleich), A_β42 (42 aa, 33 windows): largest difference per window 7e-7, GPU 4.4 s vs. 43.4 s CPU. The strongest windows lie on the known aggregation cores: GLMVGGVVIA 0.91 (C terminus), KLVFFAEDVG 0.90 (KLVFF, residues 16–20); the weakest is the N terminus DAEFRHDSGY 0.07.

```
cd ~/iver_sim/ol_amyloid
PY=~/.mamba/envs/iver/bin/python
$PY amylo_gpu.py --import-tsv kontrolle.tsv --rolle kontrolle # create candidates (acc, short name, name)
$PY amylo_gpu.py --kandidaten amyloid --bemerkung "43 amyloid candidates"
$PY amylo_gpu.py --acc YP_009724390.1 -w 10
```

8.4 Tables in wagodb (prefix amy1_)

Schema ~/iver_sim/ol_amyloid/amy1_schema.sql. Every fact is stored exactly once: candidate → sequence version → result per run → window. Window texts, window count and NCBI title/taxon are not stored but derived in the views. The candidate list lives in amy1_kandidat, no longer in a TSV file.

Table	Content
amy1_kandidat	once per protein: short name, name, accession without version, role (amyloid / kontrolle / einzel), mature chain from-to (NULL = whole sequence), remark
amy1_sequenz	once per sequence version: candidate, versioned accession (as in blast_acc), source, length, sequence, SHA1
amy1_lauf	per call: start, run time, host, GPU, software versions, window size, number of sequences, remark
amy1_ergebnis	per run and sequence: scored range, mean, maximum, position of the maximum, run time
amy1_fenster	per window: position (1-based, relative to the whole sequence), probability
amy1_v_ergebnis, amy1_v_fenster (views)	results with candidate, strongest window (SUBSTRING), window count, plus title and taxon from blast_acc/blast_seq/blast_taxon

Short	Accession	Length	Mean	Max	Pos.	strongest window
Alns	P01308.1	110	0.45	0.96	94	QCCTSICSLY
AApoCIII	P02656.1	99	0.36	0.96	5	VLLVVALLAL
AGel	P06396.1	782	0.34	0.96	184	VVVQRLFQVK
ALys	P61626.1	148	0.42	0.96	8	GLVLLSVTVQ
AApoAIV	P06727.4	396	0.28	0.96	5	AVVLTALVA
AIAPP	P10997.1	89	0.51	0.95	9	FLIVLSVALN
AApoAII	P02652.1	100	0.42	0.95	84	VNFLSYFVEL
AMed	Q08431.3	387	0.43	0.95	313	VQFVASYKVA
AB2M	P61769.1	119	0.41	0.95	5	VALAVLALLS
APro	P01236.1	227	0.42	0.95	108	FLSLIVSILR
ABri	Q9Y287.1	266	0.43	0.95	62	AFMLAGVILG
ALac	P02788.6	710	0.36	0.95	5	FLVLLFLGAL
APrP	P04156.1	253	0.35	0.95	240	VILLISFLIF
AApoCII	P02655.1	101	0.43	0.95	9	LFLVLLVLGF
AFib	P02671.2	866	0.28	0.95	6	IVCLVLSVVG
ACal	P01258.3	141	0.35	0.95	10	LALSILVLLQ
AANF	P01160.2	151	0.26	0.95	10	SFLLLLAFQL
CsgA	P28307.3	151	0.43	0.94	5	KVAAIAAIVF
ATTR	P02766.1	147	0.42	0.94	8	LLCLAGLVFV
ASem1	P04279.2	462	0.28	0.94	432	GGLDIVIEEQ
SOD1	P00441.2	154	0.35	0.94	145	LACGVIGIAQ
AL	P01834.2	107	0.36	0.94	24	SVVCLLNNFY
AA	P0DJ18.2	122	0.31	0.94	4	LTGLVFCSLV
ALECT2	O14960.2	151	0.38	0.93	97	GFCVKMFYIK
AaSyn	P37840.1	140	0.36	0.93	70	VVTGVTAVAQ
SEVI	P15309.3	386	0.38	0.92	258	LQGGVLVNEI
ATau	P10636.5	758	0.16	0.92	619	GGGSVQIVYK
HETS	Q03689.3	289	0.37	0.92	5	FGIVAGALNV
ACor	Q15517.4	529	0.31	0.90	16	GMMALLLAGL
AGal7	P47929.2	136	0.35	0.89	32	RFHVNLLCGE
FUS	P35637.1	526	0.31	0.88	140	QQSYGQQQSY

8.6 API and worker (amylo-api-worker)

AmyloDeep no longer depends on an SSH session: the same JSON API as for DIAMOND accepts jobs, with the same keys, the same rate limit and the same queue pattern. The reason for a dedicated worker is the model: loading it once costs 6.5 s, and every call to `amylo_gpu.py` pays that again. The worker (`systemd amylo-api-worker`, `env iwer`) keeps ESM-2 150M/650M, UniRep, SVM and XGBoost on the Tesla P4 (4.6 GB) and processes jobs one at a time.

Call	Access	Returns
POST ?r=amylo	X-API-Key	creates a job: <code>acc</code> (from <code>nr</code> , optional <code>from/to</code>) or <code>seq</code> , window 4–40 (default 10); response 202 with <code>job_id</code> and result URL
GET ?r=amylojob&id=...	anyone with the id	status, parameters, run time; when done the mean, maximum, strongest window and every window value

Limits as for the search: at most 3 open jobs and a daily quota per key, 60 requests per minute, up to 5,000 residues per job. The user `blast_api` may write `amyl_job` and only read the result tables. Every job creates its own `amyl_lauf`, and the values go into the same tables as manual runs (8.4) – so an API result stays just as analysable later.

8.7 Throughput on the P4

301 worker jobs in `amyl_lauf` carry a run time, 117,150 windows in 212 min altogether. Run time is linear in the number of windows ($R^2 = 0.9985$), so a least-squares line cleanly separates the fixed cost per job from the compute cost per window.

$$t = 0.69 \text{ s} + 0.1067 \text{ s} * \text{windows}$$

Quantity	Value
Windows/s (slope)	9.37
per window	107 ms

Quantity	Value
fixed per job	0.69 s
Windows/s per job	7.62–9.50, median 9.25
1,000-residue protein	about 107 s
Batch: 43 sequences in one call	9.18 windows/s

One window per residue position, so windows/s and residues/s are practically the same. The fixed cost per job is small because the models stay loaded in the worker – a call to `amylo_gpu.py` pays the load time from 8.6 every time instead. Batching several sequences into one call gains nothing further: the P4 is already saturated by the windows of a single sequence.

The least-squares line measures single jobs. Whether the chain of API, queue and worker also holds up under sustained load only shows in a full run: on 2026-09-23, 301 jobs ran back to back from 13:18–16:58, 301 done, 0 failed.

Quantity	Value
Wall clock	3.66 h
Compute time on the P4	3.53 h
Worker utilisation	96.4 %
Throughput	82.2 jobs/h
Time spent queued	mean 76 s, at most 150 s

At 96.4 % utilisation the queue never ran dry over 3.66 h: the bottleneck is the P4, not the API and not the submission side. No job was lost and the wait stayed below 150 s – the worker drains the queue as fast as it is filled.

Against the CPU version (section 8.3, A_β42: 4.4 s vs. 43.4 s) that is roughly a factor of ten. All figures are for windows of size 10; the P4 runs at 100 % utilisation and draws 55–65 W at 75–77 °C (nvidia-smi, 2026-09-23).

8.8 Example: checking a published claim (spike 194–203)

This case shows what the parts achieve together. The starting point is a claim from the literature (Nyström & Hammarström, *JACS* 2022, taken here as given and **not** verified): the SARS-CoV-2 spike contains seven amyloidogenic segments, and fragment **194–203** forms fibrils after cleavage by neutrophil elastase. What is tested is only whether AmyloDeep finds this segment conspicuous; the model does not predict cleavage sites.

Section 8.5 dictates the method: a high value alone says nothing, since all 43 known amyloid formers scored between 0.88 and 0.97. Two reference points are needed: **internal**, the rank of window 194 among all windows of the spike, and **external**, the distribution of windows from a control set of random human SwissProt proteins (100–800 aa, known amyloid formers excluded, fixed random seed).

`~/iver_sim/ol_amyloid/spike_amyloid_test.py` creates candidates and controls and evaluates afterwards; the computing goes through the API (8.5), with the worker holding the model ready:

```
# create the candidate and 300 SwissProt controls
python3 spike_amyloid_test.py --vorbereiten --n-kontrollen 300 --seed 42

# job via the API (section 8.5) - no SSH, no model loading
curl -X POST -H "X-API-Key: $K" -H 'Content-Type: application/json' \
  -d '{"acc":"YP_009724390.1","window":10}' \
  'https://yt.heissa.de/blast/api.php?r=amylo'
-> {"job_id":"0f478cae...","status":"queued","position":1,"url":"...r=amylojob&id=..."}

curl 'https://yt.heissa.de/blast/api.php?r=amylojob&id=0f478cae...'
python3 spike_amyloid_test.py --auswerten
```

What comes together here – and why this would take days instead of minutes without the infrastructure of the previous sections:

Step	Basis	Cost
fetch the spike sequence and 300 control sequences	local nr by accession (sections 4, 9)	~0.3 s per sequence, no download
select controls	swissprot_title.tsv locally, filtered by organism, length and title	15,603 candidates, selection reproducible via the seed
amyloid propensity per window	ESM-2 650M + ensemble on the Tesla P4 (8.1–8.3), submitted through the API (8.5)	1,264 windows in 136 s; GPU at 100 %, 78 °C

Step	Basis	Cost
store results	amy1_* in wagodb, each fact once (8.4)	evaluation in SQL instead of a pile of files
reproducibility	amy1_lauf records time, GPU, software versions, window size	every figure in this PDF traces back to one run

Result (run 2, 1273 aa, window 10): the window starting at position 194 (FKNIDGYFKI) scores 0.630, ranking 460 of 1,264 spike windows, above 64 % of the remaining segments.

Rank	Position	Window	Score
1	1218	LGFIAGLIAI	0.971
2	606	NQVAVLYQDV	0.964
3	607	QVAVLYQDVN	0.958
4	1059	GVVFLHVTYV	0.958
5	1222	AGLIAIVMT	0.953

The prediction does not support the claim. The fragment sits mid-field within its own protein, not at the top. The highest scores fall on hydrophobic stretches of the transmembrane and fusion region (1218, 1222, 1059) – the same pattern as with the 43 candidates, where 19 maxima lay in the signal peptide. Possible reasons, none of them tested here: AmyloDeep scores a window in isolation, not the behaviour of a peptide released by elastase; the cited work is experimental, not predictive; and other methods (WALTZ, AGGRESCAN, ZipperDB) may judge differently. What is established is only this: **this** model finds **this** segment unremarkable.

Against the controls: 300 human SwissProt proteins yield 115,886 windows; 23.28 % of them reach 0.630 or more. The smaller this share, the more conspicuous the fragment would be; at this share it does not stand out from the background. For scale: the strongest window of a control protein averages 0.930 and reaches 0.976 at most – a high single score is the norm here, not a finding.

9. Removing the duplicate storage

The same sequences were stored twice on the dell: as BLAST nr and as DIAMOND database. **State 2026-09-24:** the BLAST nr no longer exists, only the DIAMOND database remains; the table below and the figures on the BLAST nr describe the initial state.

Data set	Size	Location	State
~/blastdb/nr (BLAST, 176 volumes)	857 GB	system NVMe (78 % used)	frozen 2026-09-16
nr_full.dmnd (DIAMOND)	rebuild running (without titles, expected ~460 GB)	/mnt/archive (sda5)	updated by cron (5.2, 5.3)
Metadata blast_*	2 × ~52 GB + indexes (extrapolated, 6.1)	MariaDB (NVMe)	updated by cron (6.4)

Search, append and daily delta no longer need the BLAST nr (the hash arrays for duplicate detection are built). One purpose remains: **fetching the residues for an accession** (API acc, amylo_gpu.py). The metadata in MariaDB do not help here: blast_seq holds length and title but no sequence. On top of that the current state has a flaw: the cron scripts append new sequences only to the .dmnd and MariaDB, not to the BLAST nr. An accession added by the daily delta is in blast_acc, but blastdbcmd cannot find it.

9.1 Titles are stored three times

The BLAST nr is only just over half sequence: .psq 436 GB (1 byte per residue), .phr 270 GB headers, ~155 GB accession and taxon indexes (nr.pdb, an LMDB, alone is 99 GB), ~45 GB OID indexes. Reason: nr merges identical sequences but keeps every define – 1.15 billion sequences, 1.94 billion defines; an average title is ~139 bytes, an average sequence 377 residues.

The .dmnd contains the same titles once more. diamond_pipeline.sh calls blastdbcmd -entry all without -outfmt, which emits FASTA with the merged header of all identical entries, separated by ">" (spike YP_009724390.1: 29 defines, 2,634 bytes of titles for 1,273 residues). FastaFile takes the whole line as the ID (src/data/fasta/fasta_file.cpp:231), push_seq writes it in full after the residues (0xFF, residues, 0xFF, ID, NUL; src/legacy/dmnd/dmnd.

cpp). Only the accessions are needed: `accession_from_title` splits the line at `\1` or `>` (`FASTA_HEADER_SEP`, `src/util/sequence/sequence.cpp:38`) to fill the taxon lists. Roughly 120 GB of the `.dmnd` are title text (exact figure from the trailer once the build is done). The third copy is `blast_seq.title` in MariaDB (first title per sequence only).

Implemented in the rebuild of 2026-09-23 (`nr_build.py`, section 4): headers consist of the accessions only (`>acc1 >acc2 ...`). Taxon lists stay complete, the search output `sseqid` is only the first word anyway, and the API takes titles from MariaDB. This makes the `.dmnd` about 100 GB smaller; titles are then stored twice (`.phr` and `blast_seq.title`), once after the BLAST `nr` is deleted.

9.2 Why not diamond getseq

`diamond getseq` (`SequenceFile::get_seq()`, `src/data/sequence_file.cpp:395`) has no index: its loop reads all `sequence_count()` records in order and prints the selected ones. `--seq` expects record numbers, not accessions; title lists are also only compared during this pass. For a single accession that means reading 495 GB – hours instead of milliseconds.

9.3 Direct access: `dmnd_getseq.py`

The `.dmnd` has everything needed for direct access: from `pos_array_offset` (header) the trailer holds the offset table, 16 bytes per record (`SeqInfo: pos u64, seq_len u32`, `src/data/sequence_file.h:130`). Planned script, plain Python, no DIAMOND call:

1. Accession → OID via `blast_acc` (index `idx_acc`). Covers initial import, release append and daily delta.
2. OID → record number in the `.dmnd`: OID minus the number of empty sequences with a smaller OID. `nr_build.py` records the empty sequences in `blast_dmnd_luecke` during the build; appended sequences create no new gaps, `--append` rejects length 0 (5.1).
3. Record number → offset: one seek to `pos_array_offset + 16·n`, read 16 bytes.
4. Offset → residues: one seek, read `seq_len` bytes, translate DIAMOND's letter code back into amino acids.
5. Check on every access: the record's header follows the residues (9.1); the requested accession must appear in it, and `seq_len` must equal `blast_seq.len`. Otherwise the gap table is wrong – error instead of a wrong sequence.

Cost: two random reads on `sda5`, milliseconds. The sequences always reflect the updated `.dmnd`, including accessions from the daily delta. The offset table changes during an append; the script therefore takes the same shared lock `/home/gh/.nr_dmnd.lock` as the API worker.

9.4 Migration and release

1. `dmnd_getseq.py` built (offset table, `blast_dmnd_luecke`, `idx_acc`). Done.
2. Cross-check: against `blastdbcmd` no longer possible, the BLAST `nr` no longer exists. Instead against the 344 original sequences stored in `amyl_sequenz` (from the former BLAST `nr`, 2026-09-16): 344/344 identical with the NCBI fallback, 220/344 without (the other 124 have the same length and differ only by X masking).
3. `api.php` (`acc`) and `amylo_gpu.py` (`--acc`, `--kandidaten`) call `nr_seq.py` instead of `blastdbcmd`. Done on 2026-09-24; `api.php` runs as user `www-data`, so `nr_seq.py` sets `HOME` itself.
4. `~/blastdb` is empty, the BLAST `nr` is gone.
5. `nr_full.dmnd` lives on the system NVMe: `/home/gh/diamond/nr_full.dmnd` is the authoritative file. Worker, `diamond_append.py`, `nr_daily_delta.py`, `dmnd_getseq.py` and `blast_release.dmnd_datei` point to it; the copy on `/mnt/archive` (`sda5`) is named `NICHT_VERWENDEN_nr_full.dmnd.alt_nutze_home_gh_diamond` so it is not used by mistake. The hash cache `nr_seq_hashes.npy` still lives on `/mnt/archive`.

9.5 X masking and NCBI fallback (`nr_seq.py`)

`makedb` hard-masks low-complexity regions to X and the original is not in the `.dmnd` (measured: 1.97 % of residues, 14.5 % of sequences). The intended recovery table `blast_mask` (`nr_mask_build.py`) does not exist: it needs a pass through the BLAST `nr`, which no longer exists. `nr_seq.py` solves this per request: accession → OID (`blast_acc`), record from the `.dmnd`. If the sequence contains an X, or `blast_acc` does not know the accession, the original comes

from NCBI `efetch` (one accession, a few kB, 40 s limit, at most 3 requests per second). Only a sequence of equal length that is identical outside the X positions is accepted; otherwise the masked sequence stays and `stderr` warns. Call: `nr_seq.py P02766.1 --range 20-70` (1-based, both ends inclusive); as a module `aus_nr(acc)`.

9.6 Memory budget of the search (64 GB RAM)

The dell has 64 GB RAM, about 29 GB are permanently in use (after shutting down unused services; MariaDB with its 8.6 GB buffer pool is one of the larger ones). A search against full `nr` with the default `--block-size 2` holds 15 to 17 GB RSS; the budget for everything together is 48 GB. A two-stage search is planned: (1) a MariaDB `SELECT` picks candidates so that the memory need stays below 48 GB (intended: taxonomy rings via `blast_acc.taxid`, from the query's species outwards), (2) DIAMOND searches the subset; if it yields fewer than `max_target_seqs` hits, the next ring follows until everything is found. Caution: results are only as complete as the ring they lie in; the last ring is the full `nr` with `--block-size` as a memory cap. First `--block-size` alone is measured (script `diamond_blocksize_test.sh`, query TTR 20–70).

Implemented (2026-09-30): instead of taxonomy rings, the pre-selection by taxon, length and creation date (section “Pre-selection via MariaDB”) and the memory planning per search (section “Memory planning for every search”): limit from the memory actually free, `--block-size` chosen to fit.

9.7 Testing the API (external)

A test only counts if it runs from outside: `ssh gh@heissa.de curl 'https://yt.heissa.de/blast/api.php?r=...'`. From the dell or a workstation `yt.heissa.de` does not resolve; a call to `localhost` with `--resolve` tests only the server, not the path to it.

10. Possible next step

If DIAMOND is not sensitive enough for very distant homologues: a two-stage search over protein embeddings, reusing the pattern already in production for the knowledge base (`vec_` tables in `wagodb`, MariaDB VECTOR with HNSW index, embedding server on the P4): store sequences as vectors, prefilter with `VEC_DISTANCE_COSINE`, then check only the candidates exactly with DIAMOND/BLAST. The ESM-2 650M environment from section 8 already produces such embeddings.